Отчёт по лабораторной работе №4

Дисциплина: Операционные системы

Лебеденко Елена Викторовна

Содержание

1 1. Цель работы **5**

2 2. Задание **6**

3 3. Теоретическое введение **7**

4 4. Выполнение лабораторной работы **8**

4.1 Установка программного обеспечения 8

4.2 Практический сценарий использования git 9

4.3 Работа с репозиторием git 11

5 5. Выводы **13**

Список литературы **14**

Список иллюстраций

4.1 Установка git-flow 8
4.2 Установка Node.js 8
4.3 Настройка Node.js 9
4.4 Общепринятые коммиты 9
4.5 Репозиторий git-extended 9
4.6 Первый коммит 10
4.7 Конфигурация формата коммитов 10
4.8 Конфигурация git-flow 10
4.9 Манипуляции над релизом 1.0.0 11
4.10 Изменяю релиз 11
4.11 Релиз на github 11
4.12 Работа с репозиторием git 12

Список таблиц

1 1. Цель работы

Получение навыков правильной работы с репозиториями git.

2 2. Задание

- Выполнить работу для тестового репозитория.
- Преобразовать рабочий репозиторий в репозиторий с git-flow и conventional commits.

3 3. Теоретическое введение

- Gitflow Workflow опубликована и популяризована Винсентом Дриссеном.
- Gitflow Workflow предполагает выстраивание строгой модели ветвления с учётом выпуска проекта.
- Данная модель отлично подходит для организации рабочего процесса на основе релизов.
- Работа по модели Gitflow включает создание отдельной ветки для исправлений ошибок в рабочей среде.
- Последовательность действий при работе по модели Gitflow:
  - Из ветки master создаётся ветка develop.
  - Из ветки develop создаётся ветка release.
  - Из ветки develop создаются ветки feature.
  - Когда работа над веткой feature завершена, она сливается с веткой develop.
  - Когда работа над веткой релиза release завершена, она сливается в ветки develop и master.
  - Если в master обнаружена проблема, из master создаётся ветка hotfix.
  - Когда работа над веткой исправления hotfix завершена, она сливается в ветки develop и master.

4 4. Выполнение лабораторной работы

4.1 Установка программного обеспечения

Установка git-flow

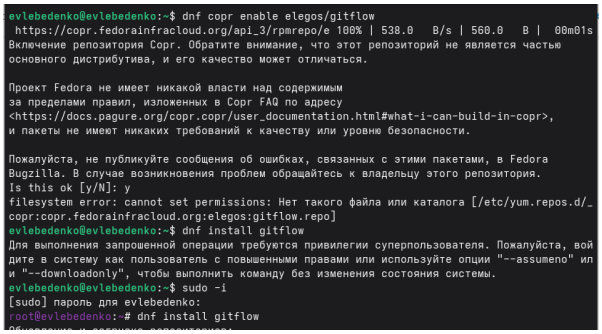

Рис. 4.1: Установка git-flow

Установка Node.js

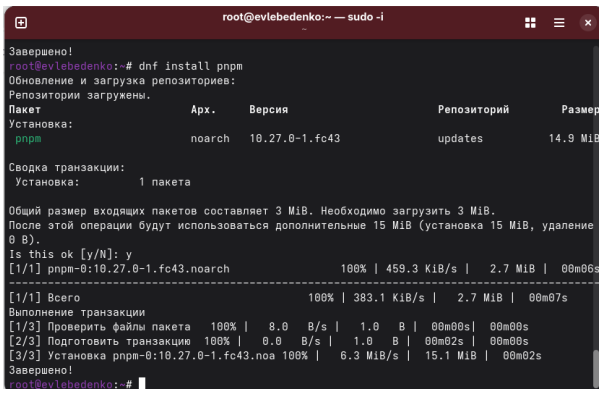

Рис. 4.2: Установка Node.js

Настройка Node.js

```
root@evlebedenko:~# pnpm setup
Appended new lines to /root/.bashrc

Next configuration changes were made:
export PNPM_HOME="/root/.local/share/pnpm"
case ":$PATH:" in
  *":$PNPM_HOME:"*) ;;
  *) export PATH="$PNPM_HOME:$PATH" ;;
esac

To start using pnpm, run:
source /root/.bashrc
root@evlebedenko:~# source ~/.bashrc
root@evlebedenko:~#
```

Рис. 4.3: Настройка Node.js

Общепринятые коммиты

```
WARN  Request took 14454ms: https://registry.npmjs.org/is-unicode-supported
WARN  Request took 15586ms: https://registry.npmjs.org/lines-and-columns
WARN  2 deprecated subdependencies found: glob@7.2.3, inflight@1.0.6
Packages: +150
++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
WARN  GET https://registry.npmjs.org/ieee754/-/ieee754-1.2.1.tgz error (ERR_SOCKET_TIMEOUT
). Will retry in 10 seconds. 2 retries left.
Progress: resolved 150, reused 0, downloaded 150, added 150, done

/root/.local/share/pnpm/global/5:
+ commitizen 4.3.1

Done in 1m 57.6s using pnpm v10.27.0
root@evlebedenko:~# pnpm add -g standard-changelog
WARN  2 deprecated subdependencies found: glob@7.2.3, inflight@1.0.6
Packages: +30
++++++++++++++++++++++++++++++
Progress: resolved 180, reused 150, downloaded 30, added 30, done

/root/.local/share/pnpm/global/5:
+ standard-changelog 7.0.1

Done in 10.6s using pnpm v10.27.0
```

Рис. 4.4: Общепринятые коммиты

4.2 Практический сценарий использования git

Подключение репозитория к github

Создаю репозиторий на GitHub. Называю его git-extended. Делаю первый коммит и выкладываю на github

```
root@evlebedenko:~# gh repo create --public git-extended
✓ Created repository Lebeduha21/git-extended on github.com
  https://github.com/Lebeduha21/git-extended
root@evlebedenko:~# git clone --recursive git@github.com:Lebeduha21/git-extended.git
Клонирование в «git-extended»...
warning: Похоже, что вы клонировали пустой репозиторий.
```

Рис. 4.5: Репозиторий git-extended

```
root@evlebedenko:~/git-extended# git add README.md
root@evlebedenko:~/git-extended# git commit -m "first commit"
[main 1b6346d] first commit
 1 file changed, 1 insertion(+)
 create mode 100644 README.md
root@evlebedenko:~/git-extended# git remote add origin git@github.com:Lebeduha21/git-extend
ed.git
error: внешний репозиторий origin уже существует
root@evlebedenko:~/git-extended# git push -u origin master
error: src refspec master ничему не соответствует
error: не удалось отправить некоторые ссылки в «github.com:Lebeduha21/git-extended.git»
root@evlebedenko:~/git-extended#
```

Рис. 4.6: Первый коммит

Конфигурирую формат коммитов, добавляю новые файлы, выполняю коммит и отправляю его на github.

```
root@evlebedenko:~/git-extended# nano package.json
root@evlebedenko:~/git-extended# git add .
root@evlebedenko:~/git-extended# git cz
cz-cli@4.3.1, cz-conventional-changelog@3.3.0

? Select the type of change that you're committing: docs:     Documentation only changes
? What is the scope of this change (e.g. component or file name): (press enter to skip)
? Write a short, imperative tense description of the change (max 94 chars):
 (29) improve documentation changes
? Provide a longer description of the change: (press enter to skip)

? Are there any breaking changes? No
? Does this change affect any open issues? No
[main 04666bb] docs: improve documentation changes
 1 file changed, 15 insertions(+)
 create mode 100644 package.json
```

Рис. 4.7: Конфигурация формата коммитов

Конфигурация git-flow

```
root@evlebedenko:~/git-extended/gitflow — sudo -i
Fatal: Missing branch name
root@evlebedenko:~/git-extended/gitflow# git flow init

Which branch should be used for bringing forth production releases?
   - develop
Branch name for production releases: [] master

Which branch should be used for integration of the "next release"?
   - develop
Branch name for "next release" development: [develop]

How to name your supporting branch prefixes?
Feature branches? [feature/]
Bugfix branches? [bugfix/]
Release branches? [release/]
Hotfix branches? [hotfix/]
Support branches? [support/]
Version tag prefix? [] v
Hooks and filters directory? [/root/git-extended/gitflow/.git/hooks]
root@evlebedenko:~/git-extended/gitflow# git branch
* develop
  master
root@evlebedenko:~/git-extended/gitflow# git push --all
Username for 'https://github.com': Lebeduha21
Password for 'https://Lebeduha21@github.com':
```

Рис. 4.8: Конфигурация git-flow

Провожу манипуляции с релизом с версией 1.0.0

```
returned error: 403
root@evlebedenko:~/git-extended/gitflow# git branch --set-upstream-to=origin/develop de
p
branch 'develop' set up to track 'origin/develop'.
root@evlebedenko:~/git-extended/gitflow# git flow release start 1.0.0
Переключились на новую ветку «release/1.0.0»

Summary of actions:
- A new branch 'release/1.0.0' was created, based on 'develop'
- You are now on branch 'release/1.0.0'

Follow-up actions:
- Bump the version number now!
- Start committing last-minute fixes in preparing your release
- When done, run:

     git flow release finish '1.0.0'

root@evlebedenko:~/git-extended/gitflow# standard-changelog --first-release
root@evlebedenko:~/git-extended/gitflow# git add CHANGELOG.md
root@evlebedenko:~/git-extended/gitflow# git commit -am 'chore(site): add changelog'
[release/1.0.0 1dbc705] chore(site): add changelog
 1 file changed, 9 insertions(+), 602 deletions(-)
root@evlebedenko:~/git-extended/gitflow#
```

Рис. 4.9: Манипуляции над релизом 1.0.0

Создаю журнал изменений, добавляю журнал изменений в индекс, заливаю релизную ветку в основную ветку и отправляю данные на github

```
root@evlebedenko:~/git-extended# standard-changelog --first-release
root@evlebedenko:~/git-extended# git add CHANGELOG.md
root@evlebedenko:~/git-extended# git commit -am 'chore(site): add changelog'
[release/1.0.0 4f18144] chore(site): add changelog
 1 file changed, 1 insertion(+)
 create mode 100644 CHANGELOG.md
root@evlebedenko:~/git-extended# git flow release finish 1.0.
Fatal: Branch 'release/1.0.' does not exist and is required.
root@evlebedenko:~/git-extended# git flow release finish 1.0.0
Переключились на ветку «main»
Эта ветка соответствует «origin/main».
Merge made by the 'ort' strategy.
 CHANGELOG.md | 1 +
 1 file changed, 1 insertion(+)
 create mode 100644 CHANGELOG.md
Уже на «main»
Ваша ветка опережает «origin/main» на 2 коммита.
  (используйте «git push», чтобы опубликовать ваши локальные коммиты)
fatal: нет описания метки?
Fatal: Tagging failed. Please run finish again to retry.
root@evlebedenko:~/git-extended# git push --all
```

Рис. 4.10: Изменяю релиз

Создаю релиз на github

```
root@evlebedenko:~/git-extended# gh release create v1.0.0 -F CHANGELOG.md
https://github.com/Lebeduha21/git-extended/releases/tag/v1.0.0
```

Рис. 4.11: Релиз на github

4.3 Работа с репозиторием git

Создадаю ветку для новой функциональности, создаю релиз git-flow, добавляю журнал изменений в индекс, заливаю релизную ветку в основную ветку и отправляю данные на github

```
fatal: Branch 'release/1.2.3' does not exist and is required.
root@evlebedenko:~/git-extended# git flow release start 1.2.3
Branches 'develop' and 'origin/develop' have diverged.
And local branch 'develop' is ahead of 'origin/develop'.
Переключились на новую ветку «release/1.2.3»

Summary of actions:
- A new branch 'release/1.2.3' was created, based on 'develop'
- You are now on branch 'release/1.2.3'

Follow-up actions:
- Bump the version number now!
- Start committing last-minute fixes in preparing your release
- When done, run:

     git flow release finish '1.2.3'

root@evlebedenko:~/git-extended# standard-changelog
root@evlebedenko:~/git-extended# git add CHANGELOG.md
root@evlebedenko:~/git-extended# git commit -am 'chore(site): update changelog'
[release/1.2.3 e86a53b] chore(site): update changelog
 1 file changed, 2 insertions(+)
root@evlebedenko:~/git-extended# git flow release finish 1.2.3
```

Рис. 4.12: Работа с репозиторием git

5 5. Выводы

Во время выполнения лабораторной работы я получила навыки правильной работы с репозиториями git.

Список литературы